Project Midterm Deliverable

Campus Marketplace with Intelligent Search and Personalized Recommendations

Student: Pooja Bhusare **Track:** Red track **Due:** October 20, 2026

1 Assigned Project and Scope

The project is a campus marketplace where SLU students can buy and sell items such as textbooks, electronics, and furniture. Sellers can create listings with an AI-generated description, and buyers can search for items and get personalized recommendations. The two main course topics covered are server-side Web APIs with databases (FastAPI, MySQL, JWT) and keeping inventory correct when two buyers try to reserve the same item at the same time.

Exclusions: real payment processing, mobile app, seller analytics.

Feature	Label	Completion criterion	Exclusion
User auth & profiles	Essential	Register as buyer or seller, log in, get JWT; expired token → 401; duplicate email → 409.	OAuth/SSO.
Listings & inventory	Essential	Seller creates/edits/deletes own listing; two simultaneous requests for the last unit → one 201 and one 409.	Bulk CSV; image CDN.
Reservation & checkout	Essential	Buyer reserves item; checkout confirms reservation; double or expired checkout → 409.	Real payments; order history.
Search & ranking	Ambitious	95% of searches finish in under 300 ms on a dataset of 500 listings; results ranked by relevance, recency, and price.	Spell correction.
Recommendations	Ambitious	Returns 5 items based on what the user has viewed; quality compared against 5 random items and reported (E2).	Collaborative filtering.
AI description generator	Ambitious	Draft description returned within 3 s; if the AI service fails, a default text is shown instead; tested with a simulated failure.	Auto re-generation.
Redis caching	If Time Permits	Frequently viewed search results are cached; response time compared with and without caching.	Advanced cache invalidation.

2 Software Implementation Plan

Module	I/O	Logic	Reused deps.
Auth	In: email, password, name, role. Out: JWT or error.	Check email not already used; store bcrypt-hashed password and role (buyer or seller); return an access token; block requests with a missing or expired token.	FastAPI, MySQL.
Listing & AI description	In: product details, or title, category, and condition. Out: listing ID or draft description.	Check required fields; block duplicate listings; send title/category to the LLM and return the draft; return a default text if the LLM fails or exceeds 3 s.	FastAPI, LLM API.
Reservation & checkout	In: listing ID or reservation ID. Out: reservation or confirmation.	Reserve only if the item is still available, using the version check (Section 6) so that only one of two simultaneous requests succeeds. Checkout confirms a pending reservation and rejects one that is already confirmed or expired. A background job runs every minute, cancels pending reservations past `expires_at` (15 minutes), and returns each unit to stock in the same transaction.	MySQL.

Module	I/O	Logic	Reused deps.
Search & ranking	In: keyword + filters. Out: ranked page of listings.	Search with MySQL full-text; score each result by text match, recency, and closeness to prices the user has viewed; hide sold-out items.	MySQL.
Recommendations	In: user ID. Out: 5 listing IDs.	Collect the user's viewed listings; find other active listings with similar titles and descriptions (TF-IDF cosine similarity); return the top 5.	scikit-learn, MySQL.
React frontend	In: user actions. Out: rendered UI.	Protect pages that need login; show search results with filters; show recommended items on the product page; update the UI immediately after a reservation.	React.

3 Architecture and Technology Decisions

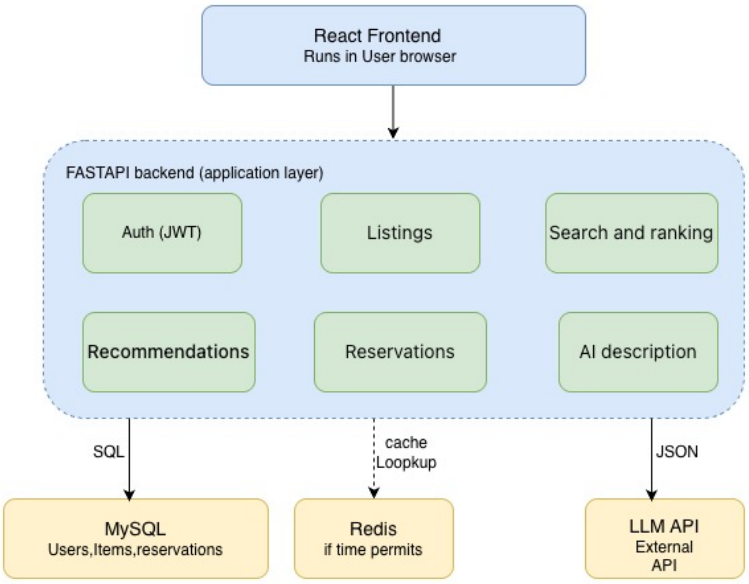


Figure 1: System architecture. The React frontend runs in the user's browser and calls the FastAPI backend over HTTPS (REST/JSON with a JWT). All backend modules run in one FastAPI application, which reads and writes MySQL over SQL, calls the external LLM API over HTTPS, and uses Redis for cache lookups only if time permits (dashed).

Tracing a reservation. When a buyer clicks *Reserve*, the browser sends `POST /reservations` with the listing ID and JWT. FastAPI verifies the token, reads the listing's quantity and version, and runs one conditional update that decrements quantity and increments version only if the version is unchanged. If the update affects one row, the API inserts a pending reservation with `expires_at` set 15 minutes ahead and returns 201; if it affects zero rows, another buyer won the race and the API returns 409. The React page then shows the reservation or a "no longer available" message.

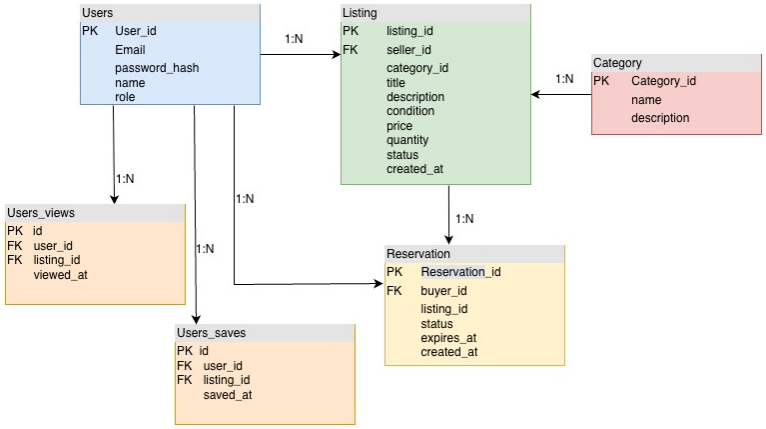


Figure 2: Data model. Arrows show one-to-many relationships: a user sells many listings and makes many reservations, a category contains many listings, and each listing can have many reservations. `Users_views` and `Users_saves` record which listings a user viewed or saved; they are the input to the recommendation module. `Listing.version` is the integer used by the reservation update to detect conflicting requests, and a unique constraint on (buyer, listing) prevents a buyer from reserving the same listing twice.

Technology choices. React 18 [3] fits a marketplace UI built from reusable product cards, filters, and live reservation updates. FastAPI [1] validates every request with Pydantic models (price, category, token), generates OpenAPI documentation used for API testing, and supports asynchronous calls, so a slow LLM request does not block other users. MySQL 8 [5] provides built-in FULLTEXT search [6], which avoids running a separate search service, and transactional conditional updates for the version-based inventory check. JWT tokens keep authentication stateless [4]. The AI description feature calls the OpenAI API [8]; it needs an API key kept in an environment variable and has a small per-request cost, which the fallback text limits. Redis would be added only if time allows, to cache search results.

4 Track-Specific Design (Red Track)

User Stories

US-1 Search and filter.
As a buyer, I want to search by keyword and filter by category and price so I only see relevant items.
Acceptance criterion: Searching "calculus" with `category=Textbooks` returns only active matching listings sorted by score; sold-out items do not appear.

US-2 Reserve an item.
As a buyer, I want to reserve an item so it is held for me.
Acceptance criterion: If two buyers reserve the last unit at the same time, exactly one gets HTTP 201 and the other gets HTTP 409; quantity is 0 after both requests complete.

US-3 Create a listing with AI description.
As a seller, I want an AI-generated description for my listing so I can write it faster.
Acceptance criterion: `POST /listings/describe` returns a draft within 3 s; if the LLM is down, a fallback template is returned instead of an error.

Interface

Figure 3 shows the search page, the starting point of the central buyer workflow: search, filter, open a listing, and reserve it.

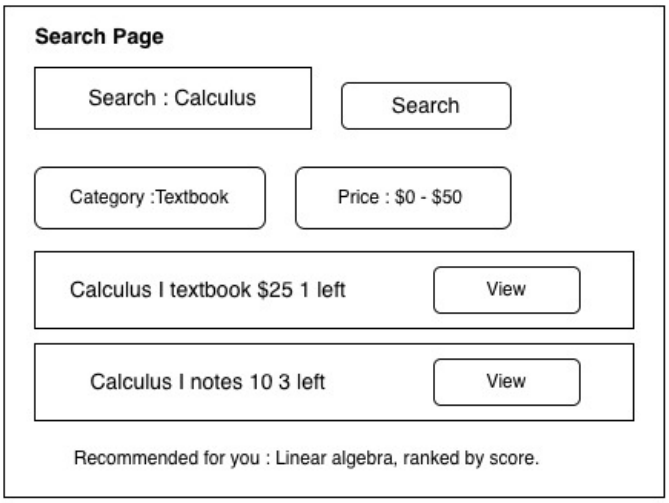


Figure 3: Wireframe of the search page: keyword box, category and price filters, ranked result cards with a View button, and a "Recommended for you" row.

API Design

Method	Endpoint / input	Success	Errors
POST	`/auth/register` {email, password, name, role}	201 {user_id}	409 duplicate email; 422 bad input.
POST	`/auth/token` {email, password}	200 {access_token}	401 wrong credentials; 422 bad input.
POST	`/listings/describe` {title, category, condition}	200 {description} within 3 s; fallback on LLM failure.	422 missing fields; 403 non-seller.
POST	`/reservations` {listing_id}	201 {reservation_id, status: "pending"}	409 sold out or already taken; 401.
GET	`/listings/search ?q=&category= &min_price=&max_price=`	200 {items, next_cursor} ranked by score; {items: []} if none.	422 invalid input.
POST	`/checkout/ {reservation_id}`	200 {status: "confirmed"}	409 already confirmed or expired; 403 not owner.

Data and Safeguards

All inputs are validated by Pydantic (price > 0, category from an allowed list, valid JWT); invalid input returns 422. Seller routes check that the user's role is seller, and buyers can confirm only their own reservations. Passwords are stored as bcrypt hashes, and the JWT signing key and LLM API key are kept in environment variables, never in the repository. If the LLM API times out or returns an error, the description endpoint catches the exception and returns a plain fallback string so the seller's flow is not blocked.

Technical Depth

Search ranking. Search goes beyond a simple keyword match. Each result is scored on three things: how well the listing text matches the query (MySQL full-text relevance [6]), how recently the listing was posted, and how close its price is to the prices the user has viewed before. The three scores are combined and results are sorted from highest to lowest.

Recommendations. When a user views a listing, it is saved to their view history. The recommendation

engine represents listing titles and descriptions as TF-IDF vectors [2], compares the user's viewed listings with all other active listings by cosine similarity, and returns the 5 most similar ones.

5 Testing and Evaluation Plan

Correctness Tests

ID	Component	Input	Expected	Mode
T1	Reservation (concurrent)	Two simultaneous `POST /reservations` for the last unit.	One 201, one 409; DB quantity = 0.	Automated.
T2	Checkout	`POST /checkout/{id}` on a pending reservation.	200; DB status = confirmed.	Automated.
T3	AI description	Mock LLM timeout; call `POST /listings/describe`.	200 with fallback text; no 500.	Automated.
T4	Search	`GET /listings/search?min_price=-5`.	422 with error detail.	Automated.
T5	Auth	`POST /reservations` with no token, then with an expired token.	401 for both.	Automated.
T6	Auth	`POST /auth/register` twice with the same email.	First 201, second 409.	Automated.

Evaluation Metrics

E1 – Search latency: Send 200 search requests, 50 at a time, against 500 listings. Target: the 95th percentile is under 300 ms. Repeat 5 times and report the median of the five runs.

E2 – Recommendation quality: Create 20 test users who each like one topic and have viewed a few items in it. Count how many of each user's top 5 recommendations match their topic, and compare with 5 random items. Target: on average at least 1.5 of 5 match (30%), clearly above random, whose expected match rate equals the share of seed listings in the user's topic (reported alongside the result). The real result is reported even if the target is missed.

E3 – No oversell: 5 buyers try to reserve each of 10 listings (quantity = 1 each) at the same time. Target: every item is reserved at most once.

Reproducibility: All tests run in Docker Compose. A SQL file pre-loads fixed test data, and results are saved to `/eval/results/`.

6 Critical Design Decision

How to Handle Two Buyers Reserving the Same Item

- **Version-based check (chosen):** the reservation update includes a check on the item's version number. If another buyer reserved it first, the version has changed, the update affects no rows, and the second request gets a 409. No waiting is needed.
- **Row lock:** lock the database row before reading (`SELECT ... FOR UPDATE` [7]), then update. Other requests wait in line until the lock is released.

Choice: version-based check. For a campus marketplace, most items have only a few buyers trying at the same time. The version-based approach has less overhead: requests never wait on a lock or hold a database connection while waiting, and a conflict is reported immediately as a 409. Test T1 and metric E3 directly verify this behavior.

Tradeoff. If a very popular item gets many requests at once, the version check rejects many of them and users see many errors. This also happens when quantity is greater than 1: two buyers who could both be served may still conflict on the version, so the loser must retry. A row lock would be better in that case.

When I would switch: if load testing shows more than 20% of requests for a popular listing failing, I

would switch that endpoint to a row lock.

AI note: Claude suggested a row lock as the default. After comparing MySQL's locking reads [7] with a version check for this workload, I chose the version-based approach because it never makes requests wait. A suggestion to use Redis as the primary inventory store was also rejected, since it would split inventory across two data stores.

7 Project Schedule, Risks, and Development Practices

Milestones

- **Oct 21 – Nov 3:** Set up the FastAPI project, MySQL schema, JWT auth, and React skeleton with login page. *Done when:* T5 and T6 pass and the app starts with Docker Compose.
- **Nov 4 – Nov 17:** Build listing CRUD, reservation with the version-based inventory check and expiry job, checkout, AI description with fallback, and ranked search. *Done when:* T1–T4 pass and search returns ranked results on 50 seed listings.
- **Nov 18 – Dec 1:** Add content-based recommendations, finish all tests, run E1 and E3. *Done when:* E3 oversell = 0 and E1 latency is recorded.
- **Dec 2 – Dec 10:** Run E2, write the report, clean up the repository and add a README; add Redis if time allows. *Done when:* E2 results are saved in `/eval/results/`, a clean Docker Compose run reproduces all results, and the PDF and repository are submitted.

Risks

Risk 1 – Too many 409s under load: if many users try to reserve a popular item at once, the version check may reject many requests. *Fallback:* add a short automatic retry on the client side; if the failure rate exceeds 20% in testing, switch to a row lock.

Risk 2 – Recommendations not better than random: TF-IDF similarity may not beat a random list on a small dataset. *Fallback:* add a simple category-match boost and report both results.

AI Usage Disclosure

I used Claude (Anthropic) to help prepare this document. It helped me organize the sections to match the assignment, review my technology choices, compare approaches for content-based recommendations, and draft and revise LaTeX tables and text. I wrote or approved all technical decisions and can explain them.

Suggestion I checked and corrected: Claude suggested a citation for optimistic concurrency that did not support my claim, so I replaced it with a source I checked myself.

Planned agent task: during development, I will ask Claude to draft the `POST /reservations` endpoint with the version-based update, given my schema and the API contract above. I will verify it by reviewing the SQL to confirm the version check and quantity decrement happen in one statement, running T1 and E3 myself, and rewriting any part I cannot explain.

References

[1] FastAPI Documentation. https://fastapi.tiangolo.com.

[2] scikit-learn. TfidfVectorizer Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[3] React Documentation. https://react.dev.

[4] Auth0. Validate JSON Web Tokens. https://auth0.com/docs/secure/tokens/json-web-tokens/validate-json-web-tokens.

[5] MYSQL documentation and reference guide. https://www.w3schools.com/mysql/default.asp.

[6] OpenAI. Text Generation Guide. https://developers.openai.com/api/docs/guides/text?lang=python.